

# Análisis y Aplicación de Metaheurísticas

Nombre del Alumno

Universidad Politécnica de Madrid, Spain

26 de octubre de 2025

## **Resumen**

# Índice

|                                                           |          |
|-----------------------------------------------------------|----------|
| <b>1. Metaheurística 1: [Nombre de la Metaheurística]</b> | <b>1</b> |
| 1.1. Introducción . . . . .                               | 1        |
| 1.2. Definiciones y Conceptos Básicos . . . . .           | 1        |
| 1.2.1. Migración . . . . .                                | 1        |
| 1.2.2. Mutación . . . . .                                 | 1        |
| 1.3. Algoritmo . . . . .                                  | 2        |
| 1.4. Implementación . . . . .                             | 2        |
| <b>Referencias</b>                                        | <b>4</b> |

# 1. Metaheurística 1: [Nombre de la Metaheurística]

## 1.1. Introducción

La Optimización Basada en Biogeografía (BBO) es un algoritmo de optimización inspirada en la biogeografía, que es el estudio de cómo se distribuyen geográficamente las especies biológicas.

## 1.2. Definiciones y Conceptos Básicos

El algoritmo BBO (Optimización Basada en Videografía) mantiene una población fija de soluciones y las modifica en cada iteración mediante dos operadores principales: Migración (1.2.1) y Mutación (1.2.2).

### 1.2.1. Migración

Supongamos que tenemos un problema y una población de soluciones candidatas representada como un vector de números enteros. Cada uno de estos números enteros es considerado un SIV (*Suitability Index Variable*). Las soluciones que son buenas son consideradas habitantes con un HSI alto y las que son malas tienen un HSI bajo. Las soluciones con un HSI alto representan hábitats de muchas especies mientras las soluciones con un HSI bajo representan hábitats de pocas especies. Supongamos dos soluciones  $S_1$  y  $S_2$  para la explicación:

- $S_1$  es considerada una solución con un HSI bajo, representa un hábitat con pocas especies. Su tasa de inmigración  $\lambda_1$  va a ser más alta que  $\lambda_2$ . Su tasa de emigración  $\mu_1$  va a ser más baja que  $\mu_2$ .
- $S_2$  es considerada una solución con un HSI alto, representa un hábitat con muchas especies. Su tasa de inmigración  $\lambda_2$  va a ser más baja que  $\lambda_1$ . Su tasa de emigración  $\mu_2$  va a ser más alta que  $\mu_1$ .

Se utilizan las tasas de emigración ( $\mu$ ) e inmigración ( $\lambda$ ) de cada solución para compartir información probabilísticamente entre hábitats<sup>15</sup>. El proceso de modificación funciona así:

- La probabilidad de que una solución  $H_i$  sea seleccionada para ser modificada es proporcional a su tasa de inmigración  $\lambda_i$ .
- Si la solución  $H_i$  es seleccionada para ser modificada, se usa la tasa de emigración  $\mu_j$  de las otras soluciones ( $j \neq i$ ) para decidir probabilísticamente cuál de ellas donará un SIV.
- Se selecciona un SIV aleatorio de la solución donante  $H_j$  y este reemplaza a un SIV aleatorio en la solución receptora  $H_i$ .

La migración en BBO es un proceso adaptativo; se usa para modificar las "islas" (soluciones) existentes<sup>19</sup>. Además, como en otros algoritmos basados en población, se incorpora elitismo para retener las mejores soluciones, evitando que sean corrompidas por la inmigración.

### 1.2.2. Mutación

Los eventos aleatorios pueden cambiar drásticamente el HSI de un hábitat. Esto se modela en BBO como la mutación del SIV, y se usan las probabilidades de conteo de especies para determinar las tasas de mutación<sup>10</sup>. Las probabilidades de las especies se rigen por la Ecuación (2).

Cada miembro de la población tiene una probabilidad asignada ( $P_s$ ), la cual hace referencia a la probabilidad que tenía a priori de existir como una solución al problema<sup>12</sup>. Tanto las soluciones con un HSI muy alto como las soluciones con un HSI muy bajo son igualmente improbables<sup>13</sup>. Las soluciones con un HSI medio son, en comparación, muy probables.

Si una solución  $S$  tiene una baja probabilidad  $P_s$ , es sorprendente que exista y, por lo tanto, es probable que mute a otra solución. Por el contrario, una solución con una probabilidad alta es menos probable

que mute. La siguiente fórmula define la tasa de mutación  $m$ , la cual es inversamente proporcional a la probabilidad de la solución:

$$m(S) = m_{\max} \left( \frac{1 - P_s}{P_{\max}} \right)$$

En esta fórmula,  $m_{\max}$  es un parámetro definido por el usuario<sup>18</sup>. Este enfoque tiende a aumentar la diversidad entre la población<sup>19</sup>. Sin él, las soluciones más probables (HSI medio) tenderían a dominar la población. Este mecanismo hace que las soluciones con HSI bajo sean propensas a mutar (dándoles la oportunidad de mejorar) y que las soluciones con HSI alto también lo sean (dándoles la oportunidad de mejorar aún más).

Es importante recalcar que se utiliza un enfoque elitista. Se guardan las mejores soluciones antes de la mutación, de modo que si una mutación de <sup>a</sup>lto riesgo <sup>a</sup>rruina el HSI de una buena solución, la original se conserva. Se espera que las soluciones con HSI medio (las más probables) mejoren a través de la migración, por lo que se evita mutarlas (aunque conservan una pequeña probabilidad de mutación).

El mecanismo de mutación implementado depende del problema<sup>24</sup>. Por ejemplo, en el problema de selección de sensores discutido en el artículo, la mutación consiste en reemplazar un sensor (SIV) escogido aleatoriamente por otro sensor generado también aleatoriamente.

### 1.3. Algoritmo

### 1.4. Implementación

La implementación del algoritmo de Optimización Basada en Biogeografía (BBO) para las funciones de prueba multiobjetivo se realizó en **Python 3**. Se utilizaron librerías como **Numpy** para cálculos numéricos vectorizados, **Matplotlib.pyplot** para generar las gráficas de los frentes de Pareto, **random** para funciones de aleatoriedad, **copy** para asegurar la creación de copias independientes de las soluciones candidatas y **scipy.special.comb** para el cálculo de coeficientes binomiales necesarios en la mutación. El código se organizó en varios componentes en un Jupyter Notebook. Primero se definieron las seis funciones de prueba ( $\mathcal{T}_1$  a  $\mathcal{T}_6$ ) siguiendo las especificaciones originales, asegurando que cada una aceptara un vector de solución y devolviera los dos objetivos  $f_1$  y  $f_2$ . El código se adaptó para manejar los dos tipos de variables de las funciones de prueba: vectores de números decimales (**float**) con sus límites correspondientes para  $\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3, \mathcal{T}_4$  y  $\mathcal{T}_6$ , así como la estructura especial de arrays de bits de longitud variable requerida por  $\mathcal{T}_5$ .

Para representar una solución individual, se creó la clase **Habitat**. Esta clase encapsula la solución (**sivs**), su valor de fitness (**hsi** - Habitat Suitability Index), y sus tasas de migración ( $\lambda$  y  $\mu$ ). El constructor (**\_\_init\_\_**) inicializa aleatoriamente los **sivs** según el **problem\_type** ('float' o 'bits'), utilizando inicialización vectorizada con **np.random.uniform** para decimales. La clase también incluye un método **clone()** para duplicar instancias de forma segura usando **copy.deepcopy()**.

La lógica principal del algoritmo BBO se implementó en la función **run\_bbo**. Dado que BBO es un algoritmo mono-objetivo, se adaptó para problemas multiobjetivo mediante una estrategia de suma ponderada: el **hsi** de cada hábitat se calculó como  $w_1 \cdot f_1 + w_2 \cdot f_2$  para un par de pesos ( $w_1, w_2$ ) dado. La función sigue el ciclo evolutivo estándar: evaluación del HSI, ordenación de la población, aplicación de elitismo (conservando los **elitism\_param** mejores), mapeo lineal de tasas de migración  $\lambda$  y  $\mu$  basado en el ranking, y los operadores de migración y mutación. Para mejorar la eficiencia, las operaciones de migración para problemas que utilizan decimales se vectorizaron utilizando NumPy, aplicando las operaciones a todos los SIVs simultáneamente mediante máscaras booleanas y **np.where**. **\*\***La mutación implementada sigue el esquema probabilístico dependiente de la probabilidad de existencia de la especie ( $P_s$ ) descrito en el BBO original. Se precalculó el vector de probabilidades de especie  $P_s$  usando una distribución binomial con  $S_{\max} = \text{pop\_size} - 1$ . Luego, para cada hábitat no elitista, se calculó una tasa de mutación individual  $m(S)$  usando la fórmula  $m(S) = \text{m\_max} \cdot (1 - P_s / P_{\max})$  y se aplicó probabilísticamente a cada SIV (vectorizado para **float**, inversión de bit para **bits**). **\*\*** Al final de las generaciones, la función devuelve los valores ( $f_1, f_2$ ) del mejor hábitat encontrado para los pesos utilizados.

Las simulaciones se realizaron adoptando un enfoque similar al de SOEA (Single-Objective Evolutionary Algorithm) con agregación. Para cada una de las seis funciones de prueba, se ejecutó **run\_bbo**

un total de 50 veces, cada vez con un par de pesos  $(w_1, w_2)$  distinto, obtenidos mediante `np.linspace` para variar  $w_1$  de 0 a 1 y calculando  $w_2 = 1 - w_1$ . Cada una de estas 50 ejecuciones se realizó durante 100 generaciones. Los 50 puntos  $(f_1, f_2)$  resultantes (uno por cada par de pesos) se recopilaron y se graficaron como un diagrama de dispersión para visualizar la aproximación del frente de Pareto. Los parámetros utilizados fueron consistentes en todas las simulaciones: tamaño de población (`POP_SIZE`) de 50, número de generaciones (`GENERATIONS`) de 100, parámetro de elitismo (`ELITISM_PARAM`) de 2, tasa máxima de migración (`MAX_MIGRATION`) de 1.0, y **tasa máxima de mutación (`m_max`) de 0.05.**

**Referencias**

**Referencias**